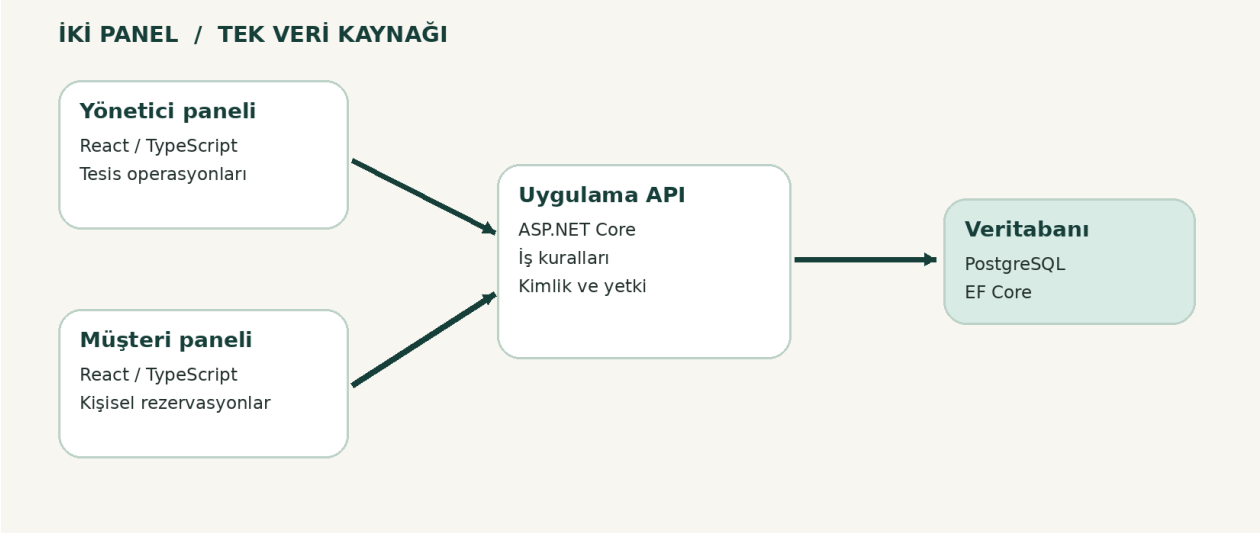
ALPINEOS / PROJE RAPORU

Kayak Merkezi Otomasyonu

Projenin çıkış noktası, yeniden tasarımı ve teknik değerlendirmesi



Hazırlayan / Fatih Yılmaz

Üniversite projesinin yeniden tasarlanmış uygulama çalışması

Özet

Bu çalışmada, üniversitede veri tabanı tasarımı üzerine geliştirdiğim Kayak Merkezi Otomasyonu fikrini web tabanlı bir uygulamaya dönüştürdüm. Hedefim yalnızca kayıt ekleyip silen bir yönetim ekranı hazırlamak değildi. Kayak merkezinin ekipman, konaklama, personel, pist, ders ve finans süreçlerini birbirine bağlı, kullanımı kolay ve daha güvenilir bir sistemde toplamak istedim. Bu nedenle yönetici paneline ek olarak müşterilerin kendi rezervasyonlarını oluşturabildiği bir portal da geliştirdim.

AlpineOS, React ve TypeScript arayüzü, ASP.NET Core API katmanı ve PostgreSQL veritabanı üzerine kuruludur. İş kurallarının sunucu tarafında uygulanması, müşteri verilerinin yetkiyle sınırlandırılması ve kiralama çakışmalarının veritabanında da engellenmesi temel tasarım kararlarımdır.

1 / Problemi nasıl tanımladım?

Bir kayak merkezinde ekipman kiralama, oda rezervasyonu, pist takibi, ders kaydı ve personel yönetimi birbirinden bağımsız görünse de gerçekte aynı müşteri, tesis ve finans verilerini paylaşır. Bu işlemlerin farklı defter veya dosyalarda izlenmesi kayıtların tekrarına, uygun olmayan kaynakların rezerve edilmesine ve işletme verisinin güncel tutulamamasına yol açabilir.

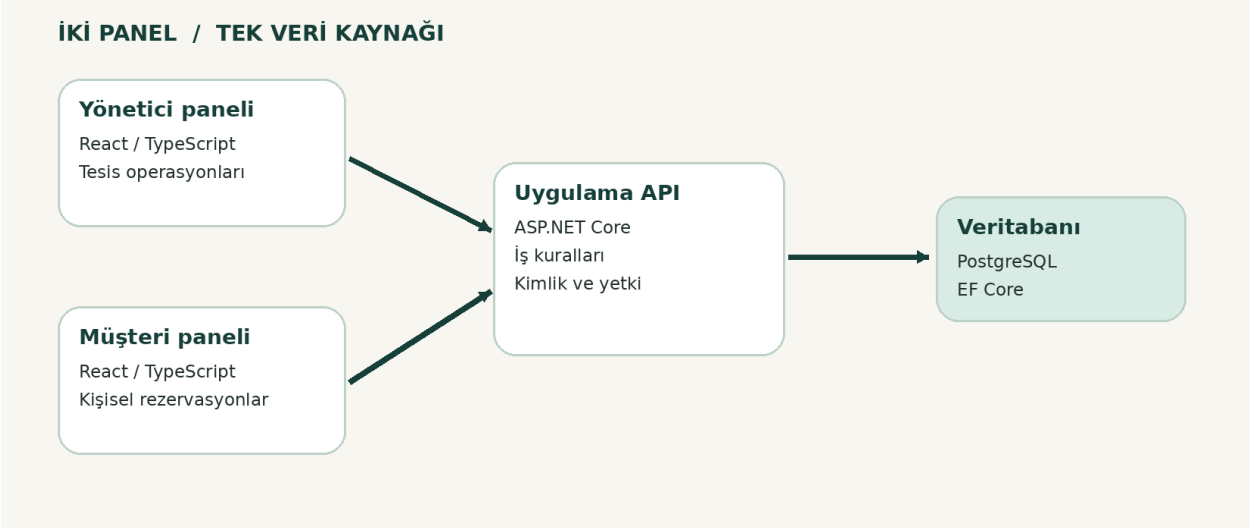
İlk projemde odak noktam veritabanı tabloları ve bunlar arasındaki ilişkilerdi. Yeniden geliştirme sırasında soruyu genişlettim: “Bu bilgileri yalnızca saklamak yerine gerçek bir işletme akışında nasıl kullanabilirim?” AlpineOS’u bu soruya cevap olarak tasarladım.

2 / Önceki projeden neleri korudum?

Eski çalışmadaki müşteri, kiralık ekipman, kiralama geçmişi, konaklama, ödeme, personel, departman, pist durumu ve etkinlik kavramlarını korudum. Orijinal arşivde Access veritabanı, rapor ve ilişki görselleri vardı. Uygulamanın eski kaynak kodları olmadığı için eski sürümde çalıştığı doğrulanamayan özellikleri yeni sistemin başarısı gibi göstermedim.

Koruduğum alan	Yeni uygulamada nasıl ele aldım?
Müşteriler	Hesap oluşturma, profil ve kişiye özel işlem geçmişi
Kiralık ekipman	Tekil demirbaş, ücret, durum ve bakım ilişkisi
Konaklama	Oda kataloğu, kapasite ve rezervasyon akışı
İşletme organizasyonu	Departman, çalışan ve rol bazlı personel hesabı
Pistler / etkinlikler	Pist durumu, dersler, yarışmalar ve katılım
Ödemeler	Gerçek para çekmeyen manuel finans kayıtları

3 / Tasarımda aldığım kararlar



Şekil 1. Yönetici ve müşteri panelleri aynı API ve veritabanına erişir.

İki ayrı program yazıp verileri birbirinden koparmak yerine tek bir API üzerinde iki farklı kullanıcı deneyimi oluşturdum. Yönetici işletmenin tamamını yönetirken müşteri yalnızca kendi profilini ve kendisine ait işlemleri görüntüler. Bu yaklaşım hem veri tutarlılığına yardımcı oluyor hem de uygulamanın bakımını kolaylaştırıyor.

3.1 Neden web tabanlı bir yapı?

Web arayüzü sayesinde uygulama farklı cihazlardan açılabiliyor. React ile her modülde tekrar kullanılan tablo, form ve kart yapıları oluşturabildim. ASP.NET Core ise veri doğrulama, yetki denetimi ve iş kurallarını tek noktada toplamak için uygun bir temel sundu.

3.2 Neden ilişkisel veritabanı?

Müşteri, kiralama, ekipman, oda ve ödeme gibi kayıtların tutarlı ilişkiler kurması gerekiyor. PostgreSQL ile bu ilişkileri daha açık tanımladım. Özellikle aynı ekipmanın veya odanın örtüşen rezervasyonlara konu olmaması için yalnızca ekrandaki kontrolü yeterli görmedim; veritabanında da kısıt kullandım.

4 / Geliştirdiğim modüller

Modül	İşlev
Yönetici kontrol paneli	Operasyon özetleri, gelir kayıtları ve tesis durumu
Müşteri ve personel	Kayıt yönetimi, departmanlar ve erişim rolleri
Ekipman ve kiralama	Katalog, rezervasyon, teslim, iade, iptal, bakım
Konaklama	Oda ve kapasite tanımı, giriş / çıkış, rezervasyon
Pist yönetimi	Pist zorluğu, kar bilgisi ve çalışma durumu
Ders / etkinlik	Kontenjan, eğitmen, katılım ve program kayıtları
Finans	Manuel tahsilat ve iade hareketleri
Müşteri portalı	Kendi hesabı, rezervasyonları, dersleri ve ödeme geçmişi

5 / Temel iş akışını nasıl oluşturdum?



Şekil 2. Ekipman ve konaklama rezervasyonlarında izlenen işlem yolu.

Müşteri önce katalogdan bir kaynak seçiyor; ardından rezervasyon aralığını ve gerekli bilgileri giriyor. Arayüz uygunluğu sorguluyor; fakat kesin kontrol, API ve veritabanı tarafında yeniden yapılıyor. Ücret bilgisi de kullanıcının gönderdiği bir tutara güvenilerek değil, kayıtlı tarife üzerinden hesaplanıyor. Oluşturulan işlem yöneticinin ekranına ve ilgili müşterinin "Rezervasyonlarım" sayfasına yansıyor.

- Kiralama sürecinde rezerve, teslim edildi, iade edildi ve iptal edildi durumları takip ediliyor.
- Konaklama sürecinde rezervasyon, giriş, tamamlanma ve iptal adımları bulunuyor.
- Müşteri, koşulları sağlayan rezervasyonunu iptal edebiliyor; ödeme kaydı olan işlemler otomatik iade edilmiyor.
- Kayak dersi ve etkinliklerde kontenjan ve yinelenen katılım kontrolleri uygulanıyor.

6 / Güvenlik ve yetki yaklaşımım

İşletme yöneticisi ile müşteri aynı işlemleri yapmamalı. Bunun için personel rollerini ayrı, müşteri erişimini ayrı ele aldım. Kimlik doğrulama çerez tabanlı; şifreler düz metin saklanmıyor. Durum değiştiren işlemlerde CSRF koruması var. Müşterinin geçmiş kayıtları API katmanında oturumdaki müşteri kimliğiyle filtreleniyor.

Bu önlemleri bir temel güvenlik katmanı olarak görüyorum. Canlı, gerçek kişisel verilerle çalışan bir sistem için ayrıca daha ayrıntılı izin denetimleri, hesap kurtarma, e-posta doğrulama, KVKK süreçleri ve bağımsız güvenlik testleri gerekir.

7 / Demo veri ve tasarımın gösterimi

Panelin boş görünmemesi ve farklı iş akışlarını gösterebilmek için kurgusal katalog kayıtları hazırladım. Demo verisi etkinse 48 ekipman, 18 konaklama birimi, 9 departman, 16 çalışan, 12 pist ve 16 etkinlik tanımı eklenebiliyor. Bunlar örnek katalog hedefleridir; mevcut veritabanındaki gerçek toplamlarla karıştırılmamalıdır.

GitHub sunumunda özellikle yönetici kontrol paneli, ekipman envanteri, müşteri keşfet sayfası, rezervasyon formu ve iki panelde aynı rezervasyonun görünmesini gösteren ekran görüntülerine yer vereceğim.

8 / Test yaklaşımı ve gözlemim

Uygulamayı Docker Compose ile yerel bilgisayarda çalıştırdım ve yönetici ile müşteri panellerini inceleyebildim. İlk frontend derleme denemesinde müşteri panelinin TypeScript veri modeliyle ilgili hata oluştu. Birleşik rezervasyon gösterimi için tanımlanan tipleri düzelttikten sonra uygulamanın çalıştığını gözlemledim.

Kaynak kodda xUnit testleri ve API kontrol betikleri de yer alıyor. Bununla birlikte her otomatik testin başarıyla geçtiğine dair kayıt saklanmadı; bu nedenle testlerin tamamlandığını iddia etmiyorum. Çakışan rezervasyon, yetkisiz erişim ve müşteri verisinin ayrılığı gibi kritik senaryoları ayrıca doğrulamak gerekiyor.

9 / Sınırlamalar ve geliştirme fikirlerim

1. Gerçek kart tahsilatı ve otomatik fatura entegrasyonu henüz bulunmuyor; finans ekranı manuel kayıt mantığında çalışıyor.
2. E-posta doğrulama ve parola sıfırlama işlevlerini tamamlamak gerekiyor.
3. Veritabanının sürümlü migration sürecine geçirilmesi ve geri yükleme provasının yapılması gerekli.
4. Tarayıcı otomasyonu, yük testleri ve üretim güvenliği kontrolleri ayrı bir doğrulama aşaması olarak planlanmalı.
5. Uygulama tek tesis odaklı; çok şubeli işletmeler için ayrıca veri izolasyonu tasarlanmalı.

10 / Sonuç

AlpineOS ile üniversitedeki veritabanı projemi daha geniş bir yazılım mühendisliği çalışmasına dönüştürdüm. En önemli kazanımım, tek tek ekranlar hazırlamaktan çok; veri modeli, backend kuralları ve iki kullanıcı türünün ihtiyaçlarını birlikte düşünmek oldu. Proje benim için ilişkisel veritabanı, C# API, React arayüzü, Docker dağıtımı ve yetkilendirme konularını bir bütün halinde uyguladığım bir çalışma haline geldi.

Kaynaklar: Orijinal Kayak Merkezi Otomasyonu arşivindeki rapor ve veri modeli görselleri; AlpineOS proje kaynak kodu, README, API dosyaları ve test betikleri.